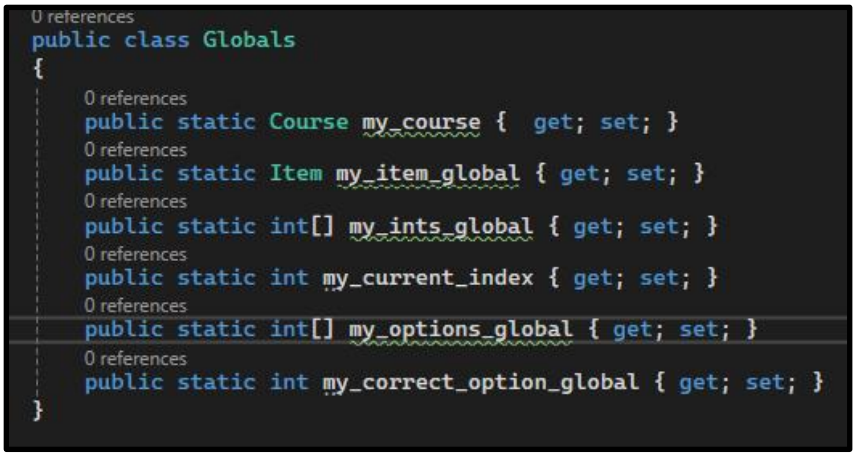
Chapter 16B: Online Learning and Testing

Welcome back to our online learning and testing example. As you will recall in our last chapter we defined the course and item classes and created a multiple-choice question display form. We also learned how to use the GroupBox, TextBox and RadioButton classes.

In this chapter we are going to randomly assign items to our multiple-choice question form, randomize the options, and finally capture the results of a submitted multiple-choice question. Let's jump to the inside and take a look.

I started by adding some project-wide globals so I didn't need to pass tons of user-defined classes between pages. I know we aren't supposed to use these guys, but everybody does so let's get use to seeing them. (Figure 16B.1)

Figure 16B.1: Global Variable Definitions



Next, I wrote a item delivery driver for multiple-choice questions. It randomizes the order of question presentation, sets up the global variables and then calls the multiple-choice test driver to display the first randomly selected item. (Figure 16B.2)

Figure 16B.2: MC_test_driver

```
1 reference
private void MC_test_driver(Item[] my_items)
{
    // randomize the questions
    int[] ints = new int[my_items.Length];
    int ints_index = -1, current_random = -1;
    Random rnd = new Random();

    for (int i = 0; i < my_items.Length; i++)
    {
        current_random = rnd.Next(my_items.Length);
        bool found_it = false;
        for (int j = 0; j < ints.Length; j++)
            if (ints[j] == current_random) found_it = true;
        if (found_it == false)
        {
            ints_index += 1;
            ints[ints_index] = current_random;
        }
    }
    Globals.my_item_global = my_items[0];
    Globals.my_ints_global = ints;
    Globals.my_current_index = ints[0];

    //Load_MC_Question_Data();
}
```

The Load_MC_Question_Data routine randomizes the answer options and then displays the my_item_global item making any answer options with zero length invisible. The figure shown here is of course only the start of this routine, but you get a sense for what is going on. Please note how I did modular arithmetic on the current number of seconds rather than use the Random.Next method. Just wanted to show you an alternative methodology. (Figure 16B.3)

Figure 16B.3: Load_MC_Question_Data

```
private void Load_MC_Question_Data()
{
    // which option will hold the correct answer
    // how many distractors do we have?
    int number_distractors;
    if (Globals.my_item_global.Distractor_2 == "") number_distractors = 1;
    else if (Globals.my_item_global.Distractor_3 == "") number_distractors = 3;
    else number_distractors = 4;

    // can't use Random because it will always come back with same sequence
    // use mod on seconds of now instead
    int currentSecond = DateTime.Now.Second;
    // add the 1 for correct answer
    int correct_answer = (currentSecond%(number_distractors + 1));

    if (correct_answer == 0) // then option A is the correct answer
    {
        Globals.my_correct_option_global = 0;
        if (number_distractors == 1)
        {
            foreach (Control c in this.Controls)
            {
                if (c is TextBox)
                {
                    TextBox textBox = (TextBox)c;
                    if (textBox.Name == "Stem") textBox.Text = Globals.my_item_global.Stem;
                    if (textBox.Name == "Option_A") textBox.Text = Globals.my_item_global.Correct;
                    if (textBox.Name == "Option_B") textBox.Text = Globals.my_item_global.Distractor_1;
                    if (textBox.Name == "Option_C") textBox.Visible = false;
                    if (textBox.Name == "Option_D") textBox.Visible = false;
                    if (textBox.Name == "Option_E") textBox.Visible = false;
                }
                else if (c is RadioButton)
                {
                    RadioButton radioButton = (RadioButton)c;
                    if (radioButton.Name == "Radio_Button_C") radioButton.Visible = false;
                    if (radioButton.Name == "Radio_Button_D") radioButton.Visible = false;
                    if (radioButton.Name == "Radio_Button_E") radioButton.Visible = false;
                }
            }
        }
        else if (number_distractors == 2)
        {
            foreach (Control c in this.Controls)
            {
                if (c is TextBox)
                {
                    TextBox textBox = (TextBox)c;
                    if (textBox.Name == "Stem") textBox.Text = Globals.my_item_global.Stem;
                    if (textBox.Name == "Option_A") textBox.Text = Globals.my_item_global.Correct;
                    if (textBox.Name == "Option_B") textBox.Text = Globals.my_item_global.Distractor_1;
                    if (textBox.Name == "Option_C") textBox.Text = Globals.my_item_global.Distractor_2;
                    if (textBox.Name == "Option_D") textBox.Visible = false;
                    if (textBox.Name == "Option_E") textBox.Visible = false;
                }
                else if (c is RadioButton)
                {
                    RadioButton radioButton = (RadioButton)c;
                    if (radioButton.Name == "Radio_Button_D") radioButton.Visible = false;
                    if (radioButton.Name == "Radio_Button_E") radioButton.Visible = false;
                }
            }
        }
        else if (number_distractors == 3)
        {
            foreach (Control c in this.Controls)
            {
                if (c is TextBox)
                {
```

Now in real-world high-stakes testing it is common to have failing candidates complain about the test or testing conditions as the reason for their failure. Nobody simply doesn't know the material, that would be way too logical. As such, we implemented a ton of measures to ensure the test and testing environment were fair and equal for everyone.

For example, the psychometricians had us record everything about the test. How many times did a candidate touch a question, did they change their answer, how much time was spent on an item, and what was their final answer were all things they had us record. In addition, we had the timer stop between questions to ensure internet speed did not influence a candidate's performance. Finally, all our testing sites had to have the same equipment and approximate testing environment. Picture IDs were checked against rosters before anybody could even sit at a terminal.

We don't know about databases yet, so no prior answers or time spent on a question will be retained in our example. As such, my setup_MC_question_data routine was pretty straightforward. Start by unchecking all the RadioButtons and then manage the previous and next buttons at the bottom of the screen. (Figure 16B.4) Please note how we used the length of our item array as the index to our my_ints_global class, pretty clever way to check for the last element in our array of classes.

Figure 16B.4: setup_MC_question_data

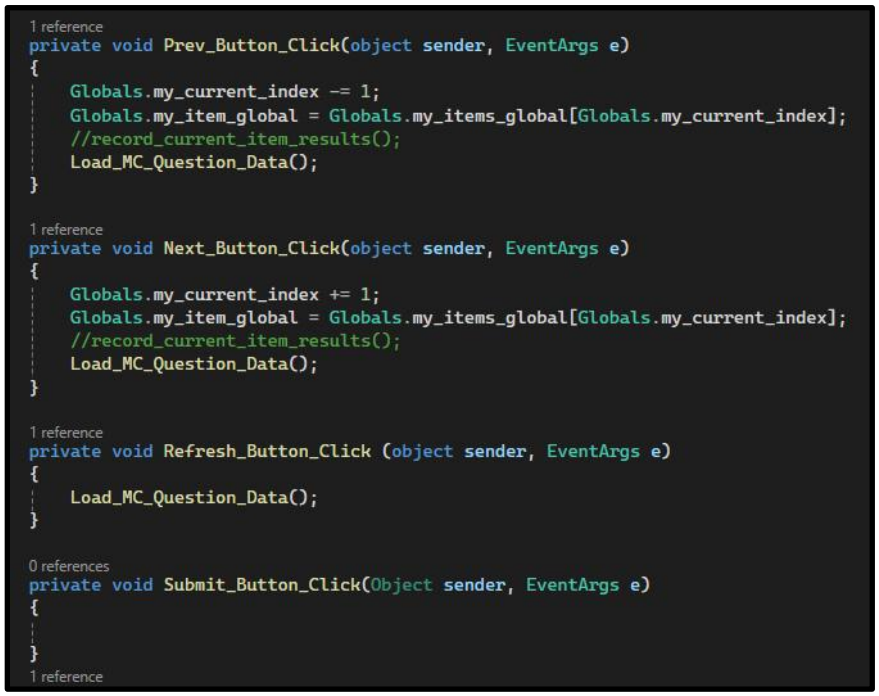
```
1 reference
private void setup_MC_question_data()
{
    // start by unchecking all RadioButtons
    foreach (Control c in this.Controls)
    {
        if (c is GroupBox)
        {
            foreach(Control c2 in c.Controls)
            {
                // all these are RadioButtons
                RadioButton rb = (RadioButton)c2;
                rb.Checked = false;
            }
        }
    }

    if (Globals.my_current_index == Globals.my_ints_global[0])
    {
        foreach (Control c in this.Controls)
        {
            if (c is Button)
            {
                Button button = (Button)c;
                if (button.Name == "Prev_Button") button.Visible = false;
            }
        }

    }
    else if (Globals.my_current_index == Globals.my_ints_global[Globals.my_ints_global.Length])
    {
        foreach(Control c  in this.Controls)
        {
            if (c is Button)
            {
                Button button = (Button)c;
                if (button.Name == "Next_Button") button.Visible = false;
            }
        }
    }
}
1 reference
private void Load_MC_Question_Data()
{
    setup_MC_question_data();
```

Next it was time to uncomment our click event handlers on those bottom buttons. Nothing interesting going on here. All we are really doing is changing our array index value.
(Figure 16B.5)

Figure 16B.5 Bottom Buttons



```
1 reference
private void Prev_Button_Click(object sender, EventArgs e)
{
    Globals.my_current_index -= 1;
    Globals.my_item_global = Globals.my_items_global[Globals.my_current_index];
    //record_current_item_results();
    Load_MC_Question_Data();
}

1 reference
private void Next_Button_Click(object sender, EventArgs e)
{
    Globals.my_current_index += 1;
    Globals.my_item_global = Globals.my_items_global[Globals.my_current_index];
    //record_current_item_results();
    Load_MC_Question_Data();
}

1 reference
private void Refresh_Button_Click (object sender, EventArgs e)
{
    Load_MC_Question_Data();
}

0 references
private void Submit_Button_Click(Object sender, EventArgs e)
{

}
1 reference
```

I am not even going to bother with the submit button as we don't have anywhere to store this stuff yet anyway and you already know how to change forms.

As such that completes our online learning and testing example. In our next chapter we are going to take a look at AI generated C# code. Some good, some bad and some just plain ugly.